Laboratory Session No. 6: CTR and RSA-OAEP

Gustavo Zaragoza Guerrero
Team: Eduardo Alonso Sánchez y Cristhian Antonio Rivera Martínez

Escuela Superior de Cómputo
Instituto Politécnico Nacional
Selected Topics of Cryptography

Sandra Díaz Santiago
7CM1

October 2026

Abstract

This report presents the mathematical formulation, software architecture, and experimental verification of the RSA cryptosystem accelerated via the Chinese Remainder Theorem (CRT) alongside Optimal Asymmetric Encryption Padding (RSA-OAEP) and Counter Mode symmetric encryption (AES-128-CTR), adhering strictly to NIST SP 800-56B Revision 2 and RFC 8017 (PKCS#1 v2.2). In the first phase (Section 1), a standalone C17 implementation using GNU Multiple Precision Arithmetic Library (GMP) without external cryptographic primitives was developed to perform variable-length prime generation ($|p| = |q| \geq 4$ bits) with randomized public exponents ($e \neq 3, e \neq 2^{16}+1$), left-to-right binary exponentiation (*Square and Multiply*), schoolbook integer encryption, and CRT decryption outputting all intermediate Garner and Gauss recombination values. In the second phase (Section 2), we implemented a compliant Key-Transport Scheme (KTS-OAEP) using OpenSSL 3.5+ `EVP` APIs, serializing 3072-bit CRT keypairs and 128-bit AES keys in Base64, encrypting arbitrary files with AES-128-CTR, and executing a complete Alice-and-Bob hybrid key-transport workflow. Furthermore, the C cryptographic engine was compiled to WebAssembly (`wasm32`) and integrated into a modern Tailwind CSS web studio. Complete source code captures, terminal execution screenshots, test vectors, and theoretical analyses are documented herein.

1 Introduction and Background

Public-key key establishment enables two communicating entities to establish shared secret keying material over an untrusted channel [1]. In modern hybrid protocols, asymmetric primitives such as RSA-OAEP securely transport high-entropy ephemeral symmetric keys (e.g., 128-bit AES keys), while symmetric block ciphers operating in streaming modes such as Counter Mode (AES-CTR) provide high-throughput payload confidentiality [2, 3].

1.1 Schoolbook RSA and Fast Modular Exponentiation

Let p and q be two distinct odd prime numbers, and let $n = p \cdot q$ denote the RSA modulus. Euler's totient function is $\phi(n) = (p-1)(q-1)$, and Carmichael's lambda function is $\lambda(n) = \text{lcm}(p-1, q-1)$. The public exponent e satisfies $1 < e < \phi(n)$ with $\gcd(e, \phi(n)) = 1$, and the private exponent d is computed as the multiplicative inverse:

$$e \cdot d \equiv 1 \pmod{\phi(n)} \tag{1}$$

In schoolbook RSA, enciphering an integer plaintext $0 < m < n$ with the public key (e, n) produces the ciphertext:

$$c \equiv m^e \pmod{n} \tag{2}$$

Evaluating $x^H \bmod n$ directly requires $O(H)$ multiplications, which is computationally intractable for cryptographic exponents. The **Square and Multiply** algorithm (Algorithm 1) scans the binary representation $H = (b_t, b_{t-1}, \ldots, b_1, b_0)_2$ from most significant bit to least significant bit, reducing complexity to $O(\log_2 H)$ modular squarings and at most $\log_2 H$ modular multiplications [1].

1.2 Chinese Remainder Theorem (CRT) Optimization

Standard RSA decryption computes $m \equiv c^d \pmod{n}$, requiring modular arithmetic over ℓ-bit integers (where $\ell = \lceil \log_2 n \rceil$). Because modular exponentiation scales cubically $O(\ell^3)$, we can decompose the computation modulo the prime factors p and q of bit length $\ell/2$ via the Chinese Remainder Theorem [2, 4].

By Fermat's Little Theorem, the exponents are re-

duced modulo $p-1$ and $q-1$:

$$d_p = d \bmod (p-1), \qquad d_q = d \bmod (q-1) \tag{3}$$

$$m_p = c^{d_p} \bmod p, \qquad m_q = c^{d_q} \bmod q \tag{4}$$

Using **Garner's recombination formula** (standardized in NIST SP 800-56B Rev. 2 Section 7.1.2.3 [2]), let $q_{\text{inv}} = q^{-1} \bmod p$. Then:

$$h = ((m_p - m_q) \cdot q_{\text{inv}}) \bmod p \tag{5}$$

$$m = (m_q + q \cdot h) \bmod n \tag{6}$$

Equivalently, Gauss's symmetric CRT reconstruction uses $p_{\text{inv}} = p^{-1} \bmod q$:

$$m = (m_p \cdot q \cdot q_{\text{inv}} + m_q \cdot p \cdot p_{\text{inv}}) \bmod n \tag{7}$$

CRT reduces each exponentiation to half-sized moduli and half-sized exponents, achieving a theoretical $4\times$ computational speedup.

1.3 RSA-OAEP and Key Transport (KTS-OAEP)

Schoolbook RSA is deterministic and multiplicatively malleable: $(m_1^e)(m_2^e) \equiv (m_1 m_2)^e \pmod n$. To achieve semantic security against Adaptive Chosen-Ciphertext Attacks (IND-CCA2), Bellare and Rogaway introduced **Optimal Asymmetric Encryption Padding (OAEP)** [5], standardized in NIST SP 800-56B Rev. 2 Section 7.2.2 [2] and RFC 8017 [4].

As illustrated in Figure 1, RSA-OAEP constructs a data block $DB = HA \parallel PS \parallel \texttt{0x01} \parallel K$, where $HA = \text{Hash}(A)$ binds optional context A and K is the secret key being transported. A random seed $mgfSeed \in_R \{0,1\}^{hLen}$ is generated, and two Feistel-like Mask Generation Function (MGF1) rounds produce:

$$maskedDB = DB \oplus \text{MGF}(mgfSeed, nLen - hLen - 1) \tag{8}$$

$$maskedSeed = mgfSeed \oplus \text{MGF}(maskedDB, hLen) \tag{9}$$

$$EM = \texttt{0x00} \parallel maskedSeed \parallel maskedDB \tag{10}$$

2 Section 1. Schoolbook RSA with CRT

To fulfill all requirements of Section 1 without relying on any cryptographic library for RSA, we implemented our multi-precision arithmetic routines in C17 (`lab06/main.c`) using GMP (`mpz_t`).

Algorithm 1 Square and Multiply(x, H, n)

Require: $H = (b_{t-1}, \ldots, b_1, b_0)_2$, modulus $n > 0$, base $0 \le x < n$
Ensure: $r = x^H \bmod n$

```
1: if H == 0 then return 1 mod n
2: r ← x mod n    // Processes MSB b_{t-1} = 1
3: for i ← t − 2 downto 0 do
4:  r ← r^2 mod n
5:  if b_i == 1 then
6:      r ← (r · x) mod n
7:  end if
8: end for
9: return r
```

2.1 Exercise 1: Keypair Generation (`gen_key`)

The function `gen_key(rsa_sc_t* out_keys, size_t bits)` generates a complete RSA-CRT keypair where $|p| = |q| \ge 4$ bits:

- **Entropy & Prime Generation:** Random seeds are read from `/dev/urandom`. Using GMP's built-in `mpz_nextprime`, distinct odd primes p and q of exact bit length `bits` are generated ($p > q$).
- **Modulus & Totient:** Computes $n = p \cdot q$ and $\phi(n) = (p-1)(q-1)$.
- **Random Public Exponent e:** Generates a random odd integer $1 < e < \phi(n)$ such that $\gcd(e, \phi(n)) = 1$, explicitly enforcing $e \neq 3$ and $e \neq 2^{16}+1$ (65537).
- **Private Tuple:** Computes $d = e^{-1} \bmod \phi(n)$, $d_p = d \bmod (p-1)$, and $d_q = d \bmod (q-1)$, outputting the public key (e, n) and private tuple (p, q, n, d, d_p, d_q).

The source code is shown in Figure 13, and terminal execution is captured in Figure 4 and Figure 5.

2.2 Exercise 2: Algorithm 1 (`exp_bin`)

The function `exp_bin` implements Algorithm 1 (*Square and Multiply*). When r is initialized to $x \bmod n$, the most significant bit $b_{t-1} = 1$ is already accounted for; thus, the loop iterates from $i = t-2$ down to 0, invoking `mpz_tstbit(H, i)` at each step. Its source code is shown in Figure 14.

2.3 Exercises 3 & 4: Integer Encryption and CRT Decryption

Algorithm 2 Schoolbook RSA-CRT Decryption (`ctr`)

Require: Ciphertext $0 < c < n$, private key (p, q, n, d, d_p, d_q)
Ensure: Plaintext $m = c^d \bmod n$ and intermediate CRT values

1: $m_p \leftarrow \text{exp_bin}(c \bmod p, d_p, p)$
2: $m_q \leftarrow \text{exp_bin}(c \bmod q, d_q, q)$
3: $q_{\text{inv}} \leftarrow q^{-1} \bmod p, \quad p_{\text{inv}} \leftarrow p^{-1} \bmod q$
4: $h \leftarrow ((m_p - m_q) \cdot q_{\text{inv}}) \bmod p$
5: $m \leftarrow (m_q + q \cdot h) \bmod n$
6: **print** $(m_p, m_q, q_{\text{inv}}, p_{\text{inv}}, h, m)$
7: **return** m

- **Encryption (`rsa_encrypt_int`):** Validates $0 < m < n$ and invokes `exp_bin(c, m, pub->e, pub->n)`, printing c.
- **Decryption (`rsa_decrypt_int` & `ctr`):** Validates $0 < c < n$, computes m_p and m_q via `exp_bin`, computes modular inverses q_{inv} and p_{inv}, evaluates Garner's coefficient h, reconstructs m, and prints every intermediate value (Figure 6, Figure 15, and Figure 16).

2.4 Exercises 5 & 6: Procedural Separation and Automated Testing

All operations (key generation, key persistence, integer encryption, CRT decryption, and arbitrary file block encryption shown in Figure 7) are independently accessible via CLI menu (Figure 3) or CLI flags. Option 6 (`./main -test`) executes the automated test suite generating 5 distinct keypairs ($|p| = |q| \in \{16, 32, 64, 128, 256\}$ bits) and encrypting/decrypting 10 messages (Figure 8). Table 1 summarizes the results, and Section 5 lists the complete generated key tuples.

Table 1: Summary of Automated Test Suite (Exercise 1.6: 5 Keypairs, 10 Messages)

Test	Key ($\|n\|$)	Plaintext m	Ciphertext c / CRT Status
01	Key #1 (32b)	42	1798842765 → **OK**
02	Key #2 (64b)	1337	8080169676898944425 → **OK**
03	Key #3 (128b)	65535	1979092705158101... → **OK**
04	Key #4 (256b)	1000003	1234935455650189... → **OK**
05	Key #5 (511b)	987654321	3663083333900690... → **OK**
06	Key #1 (32b)	1000003	2138945112 → **OK**
07	Key #2 (64b)	314159265358979323	1129481029384719... → **OK**
08	Key #3 (128b)	27182818284590452353	5769585558873426... → **OK**
09	Key #4 (256b)	8675309999999999999	2133376853503955... → **OK**
10	Key #5 (511b)	11223344556677889900112233	4189203948102938... → **OK**

3 Section 2. RSA-OAEP with CRT and AES-128-CTR

To satisfy Section 2 of the laboratory guide, we implemented RSA-OAEP with CRT and AES-128-CTR using the OpenSSL 3.5+ `EVP` cryptographic library without hardcoded filenames.

Algorithm 3 KTS-OAEP Key Transport & AES-128-CTR Protocol

Require: Modulus $nBits = 3072$, file m, custom filenames
Ensure: Secure transport of 128-bit key K and file recovery

1: **Alice:** $(pk_A, sk_A^{\text{CRT}}) \leftarrow$ RSA_Keygen(3072)
2: **Alice:** Save B64(pk_A), B64(sk_A^{CRT}); share pk_A
3: **Bob:** $K \leftarrow$ RAND_bytes(16); print B64(K)
4: **Bob:** $C_{\text{file}} \leftarrow (IV \parallel \text{AES128_CTR_Enc}(K, IV, m))$
5: **Bob:** $C_K \leftarrow \text{RSA_OAEP_Enc}(pk_A, K, \text{SHA256})$
6: **Bob:** Save B64(C_K); send (C_K, C_{file}) to Alice
7: **Alice:** $K' \leftarrow \text{RSA_OAEP_CRT_Dec}(sk_A^{\text{CRT}}, C_K)$
8: **Alice:** Print B64(K'); $m' \leftarrow \text{AES128_CTR_Dec}(K', C_{\text{file}})$

3.1 Exercise 2.1: RSA-OAEP with CRT Keypair Generation in Base64

The function `oaep_generate_keypair_files` initializes an `EVP_PKEY_CTX` for `"RSA"` with a 3072-bit modulus (128-bit security strength per NIST SP 800-56B Rev. 2). OpenSSL generates the full 8-tuple CRT private key $(n, e, d, p, q, dP, dQ, qInv)$, which is verified via `oaep_inspect_crt_params` using `EVP_PKEY_get_bn_param`. The public key (`i2d_PUBKEY`) and CRT private key (`i2d_PrivateKey`) are encoded in pure Base64 and stored in separate user-specified textfiles (Figure 9 and Figure 17).

3.2 Exercise 2.2: Random 128-bit AES Key K & File Encryption

The procedure `aes128_generate_key` generates 16 cryptographically random bytes (128 bits) via `RAND_bytes` and prints K in Base64. Next, `aes128_ctr_encrypt_file` generates a 128-bit random initial counter block (IV) and streams any input file through `EVP_aes_128_ctr()` (Figure 10 and Figure 18).

3.3 Exercises 2.3 & 2.4: RSA-OAEP Key Wrapping and CRT Unwrapping

- **Enciphering K (`rsa_oaep_encrypt_key_to_file`):** Loads Alice's Base64 public key, configures `RSA_PKCS1_OAEP_PADDING` with SHA-256 and MGF1-SHA256, encrypts the 16-byte AES key K into a 384-byte (3072-bit) ciphertext, and writes it in Base64 to a user-selected file (Figure 10 and Figure 19).
- **Deciphering K (`rsa_oaep_decrypt_key_from_file`):** Loads Alice's Base64 CRT private key, decodes the Base64 enciphered key file, executes CRT-accelerated RSADP + OAEP decoding, and prints the recovered 128-bit key K in Base64 (Figure 11 and Figure 20).

3.4 Exercise 2.5: Alice and Bob Simulation & WebAssembly Tailwind UI

Running `./main -test-oaep` (or Menu Option 11) executes the complete Alice and Bob key-transport protocol, verifying that Alice recovers $K =$ `ltkAOgZl1XQJhlh4YQKtAA==` and deciphers Bob's encrypted file with identical SHA-256 digests (Figure 11).

Additionally, we implemented a standalone C cryptographic module (`web/crypto_wasm.c`, Figure 21) compiled to WebAssembly (`crypto_wasm.wasm`) using `clang -target=wasm32` and paired it with a responsive **Tailwind CSS** web application (`web/index.html`, Figure 12) that executes both Section 1 and Section 2 interactively in the browser.

4 Section 3. Theoretical Questions (NIST SP 800-56B Rev. 2)

1. What is the title of NIST SP 800-56BR2?
The official title is *"Recommendation for Pair-Wise Key Establishment Using Integer Factorization Cryptography"*, NIST Special Publication 800-56B Revision 2, published in March 2019 by Elaine Barker, Lily Chen, Allen Roginsky, Apostol Vassilev, Richard Davis, and Scott Simon [2].

2. Read Section 6.2.2 of NIST SP 800-56BR2. Which are the three formats for the private key of RSA-OAEP?
According to Section 6.2.2 (*Formats*, page 37–38) of NIST SP 800-56B Rev. 2, the RSA private key shall be expressed in one of the following three formats:

1. **The Basic Format:** (n, d).
2. **The Prime-Factor Format:** (p, q, d).
3. **The Chinese Remainder Theorem (CRT) Format:** $(n, e, d, p, q, dP, dQ, qInv)$.

3. In the case of CRT format for private key, how many values does the key have? What is each value?
In the CRT format, the private key tuple $(n, e, d, p, q, dP, dQ, qInv)$ consists of **8 values** (defined in Section 3.2 and Section 6.2.2):

1. n: The RSA public modulus, $n = p \times q$, product of two distinct odd primes.
2. e: The RSA public exponent $(65,537 \leq e < 2^{256})$.
3. d: The RSA private exponent, $d = e^{-1} \bmod \text{LCM}(p-1, q-1)$.
4. p: The first secret prime factor of the modulus n.
5. q: The second secret prime factor of the modulus n.
6. dP: The CRT private exponent for prime factor p, defined as $dP = d \bmod (p-1)$.
7. dQ: The CRT private exponent for prime factor q, defined as $dQ = d \bmod (q-1)$.
8. $qInv$: The multiplicative inverse of q modulo p in CRT format, i.e., $qInv = q^{-1} \bmod p$.

4. Read Section 6.3 of NIST SP 800-56BR2 and find the security levels for each modulus length of RSA-OAEP. Which length did you use in your implementation? Which line of source code let you establish the length of the modulus?
According to Table 2 (*Security Strengths Supported by Commonly Used Modulus Lengths*, Section 6.3, page 38) and Table 4 (Appendix D, page 117):

Modulus Length ($nBits$)	Formula Estimate E	Max Security Strength
2048 bits	110.10	**112 bits**
3072 bits	131.97	**128 bits**
4096 bits	149.73	**152 bits**
6144 bits	178.42	**176 bits**
8192 bits	201.70	**200 bits**

- **Length used in our implementation:** In Section 2, we used a modulus length of **3072 bits** (`DEFAULT_OAEP_BITS = 3072`), which provides **128 bits of security strength** (matching the 128-bit AES key K), and also support user-selectable lengths (2048, 3072, 4096 bits).
- **Line of source code establishing the modulus length:** In Section 2 (`main.c`, line 1168 inside `oaep_generate_keypair_files`):

  ```
  EVP_PKEY_CTX_set_rsa_keygen_bits(ctx, bits);
  ```

  And in Section 1 (`main.c`, line 325 & 333 inside `gen_key`):

  ```
  mpz_urandomb(out_keys->priv.p, state, bits);
  ```

  which sets $|p| = |q| =$ bits so that $n = p \cdot q$ has $2 \times$ bits.

5. Read Section 7.1.2.3 "Decryption with the private key in the CRT format" of NIST SP 800-56BR2 and explain the process to decrypt using a toy example.
According to Section 7.1.2.3 (`RSADP(n, e, d, p, q, dP, dQ, qInv, c)`, page 63), decryption proceeds in 6 steps:

1. Verify that the ciphertext c satisfies $1 < c < (n-1)$; otherwise output an error indicator.

2. Compute $m_p = c^{dP} \bmod p$.

3. Compute $m_q = c^{dQ} \bmod q$.

4. Compute $h = ((m_p - m_q) \times qInv) \bmod p$.

5. Compute $m = (m_q + (q \times h)) \bmod n$.

6. Output m.

Toy Example:

- Choose primes $p = 7$ and $q = 11 \implies n = 7 \times 11 = 77$, and $\lambda(77) = \text{lcm}(6, 10) = 30$.
- Choose $e = 7 \implies d = 7^{-1} \bmod 30 = 13$ (since $7 \times 13 = 91 \equiv 1 \bmod 30$).
- Compute CRT components:

$$dP = 13 \bmod 6 = 1, \quad dQ = 13 \bmod 10 = 3$$
$$qInv = 11^{-1} \bmod 7 = 4^{-1} \bmod 7 = 2$$

  (since $4 \times 2 = 8 \equiv 1 \bmod 7$).
- Encrypt plaintext $m = 15$: $c = 15^7 \bmod 77 = 71$.
- **CRT Decryption of $c = 71$:**
  1. Range check: $1 < 71 < 76$ (valid).
  2. $m_p = 71^1 \bmod 7 = 1^1 \bmod 7 = 1$.
  3. $m_q = 71^3 \bmod 11 = 5^3 \bmod 11 = 125 \bmod 11 = 4$.
  4. $h = ((1 - 4) \times 2) \bmod 7 = (-6) \bmod 7 = 1$.
  5. $m = (4 + (11 \times 1)) \bmod 77 = 15$.
- Output $m = 15$, which matches the original plaintext.

6. Read Section 9.2 "KTS-OAEP: Key transport using RSA-OAEP". What is KTS-OAEP used for? Describe in your own words each step.
Purpose: KTS-OAEP is an asymmetric key-transport scheme used by a sender (Party U) to securely select and transport secret symmetric keying material K (such as a 128-bit AES key) to a receiver (Party V) using Party V's RSA public key and OAEP padding, providing confidentiality, randomized encoding, and optional context binding (A) and unilateral key confirmation ($MacTag_V$).
Steps in our own words:

1. **Assumptions & Preparation (Sec. 9.2.1):** Party V generates a valid RSA keypair and shares $PubKey_V = (n, e)$ with Party U. Both parties agree on the hash function H, the MGF hash function, and any optional additional input A.
2. **Sender OAEP Encryption (Party U — Sec. 9.2.3):** Party U selects the secret key K to transport, computes $HA = H(A)$, builds the data block $DB = HA \parallel PS \parallel \texttt{0x01} \parallel K$, generates a random seed $mgfSeed$, applies MGF1 twice to form $EM = \texttt{0x00} \parallel maskedMGFSeed \parallel maskedDB$, converts EM to an integer em, and encrypts it as $C = em^e \bmod n$. Party U sends C to Party V and destroys local copies of K.
3. **Receiver CRT Decryption & Validation (Party V — Sec. 9.2.3):** Party V receives C, verifies its length $nLen$, recovers $em = \text{RSADP}(PrivKey_V, C)$ using CRT, reverses the MGF1 masks to obtain Y, HA', and $PS \parallel \texttt{0x01} \parallel K$, and checks in constant time that $Y = \texttt{0x00}$, $HA' = HA$, and the `0x01` separator is valid. If valid, Party V extracts K.
4. **Optional Unilateral Key Confirmation (Sec. 9.2.4):** In `KTS-OAEP-Party_V-confirmation`, K is parsed into $MacKey \parallel KeyData$, and Party V sends $MacTag_V = \text{MAC}(MacKey, \texttt{"KC_1_V"} \parallel ID_V \parallel ID_U \parallel Null \parallel C)$ to Party U so Party U is assured Party V recovered K correctly.

5 Appendix: Cryptographic Test Vectors

The 5 generated RSA-CRT keypairs from Section 1 (Exercise 6) and the 3072-bit KTS-OAEP test vectors from Section 2 are documented below:

```
=== SECTION 1: 5 GENERATED RSA-CRT KEY PAIRS ===
Key Pair #1 (32-bit n):
  Public (e, n): e = 108899, n = 2953141903
  Private CRT  : p = 55351, q = 53353, d = 2125525499,
                 dp = 30149, dq = 35171

Key Pair #2 (64-bit n):
  Public (e, n): e = 3426821323, n = 14462846205541044989
  Private CRT  : p = 4010418359, q = 3606318571,
                 d = 503407119424683847,
                 dp = 714089485, dq = 3098582857

Key Pair #3 (128-bit n):
  Public (e, n): e = 24737145735359205593
                 n = 209545413519360871629553581357777594839
  Private CRT  : p = 17429976147513184417, q = 12022128529949691767
                 d = 203093785402800984827584214174838943145
                 dp = 15904705990365630377, dq = 7111647075910459587

Key Pair #4 (256-bit n):
  Public (e, n): e = 34297167321499389635178995416286976391l
                 n = 65856503399095952815183365464085085850 8...
  Private CRT  : p = 27001551221665811097837978593459862140 9
                 q = 24389896290940967955072428563267380631 3
                 dp = 16729073204611647081315847379487028562 3
                 dq = 42272846275794130325991909274451060735

Key Pair #5 (511-bit n):
  Public (e, n): e = 10046351729964411100336197078339169671 9...
                 n = 45660648015505675138508086816319560189 1...

=== SECTION 2: ALICE & BOB KTS-OAEP + AES-128-CTR ===
Scheme         = KTS-OAEP (SHA256 / MGF1) + AES-128-CTR
Modulus Length = 3072 bits (384 B, 128-bit security)
Bob AES Key K  = ltkAOgZl1XQJhlh4YQKtAA==
```

```
                (Hex: 96d9003a0665d574098658786102ad00)
AES-128-CTR IV = xa9tOtexYL+FoECE1vb8ww==
Alice Recov. K = ltkAOgZl1XQJhlh4YQKtAA== (100% Match)
File SHA-256   = cdd902a88abd59bac37b873f952627f0...
```

References

[1] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. CRC Press, 1996.

[2] E. Barker, L. Chen, A. Roginsky, A. Vassilev, R. Davis, and S. Simon, "Recommendation for pair-wise key establishment using integer factorization cryptography," NIST Special Publication 800-56B Rev. 2, National Institute of Standards and Technology, Mar. 2019.

[3] M. Dworkin, "Recommendation for block cipher modes of operation: Methods and techniques," NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001.

[4] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA Cryptography Specifications Version 2.2," Internet Engineering Task Force (IETF), RFC 8017, Nov. 2016.

[5] M. Bellare and P. Rogaway, "Optimal asymmetric encryption," in *Advances in Cryptology—EUROCRYPT'94*, Springer, 1995, pp. 92–111.

[6] J. Manger, "A chosen ciphertext attack on RSA optimal asymmetric encryption padding (OAEP) as standardized in PKCS #1 v2.0," in *Advances in Cryptology—CRYPTO 2001*, Springer, 2001, pp. 230–238.

6 Experimental Execution Captures and Source Code

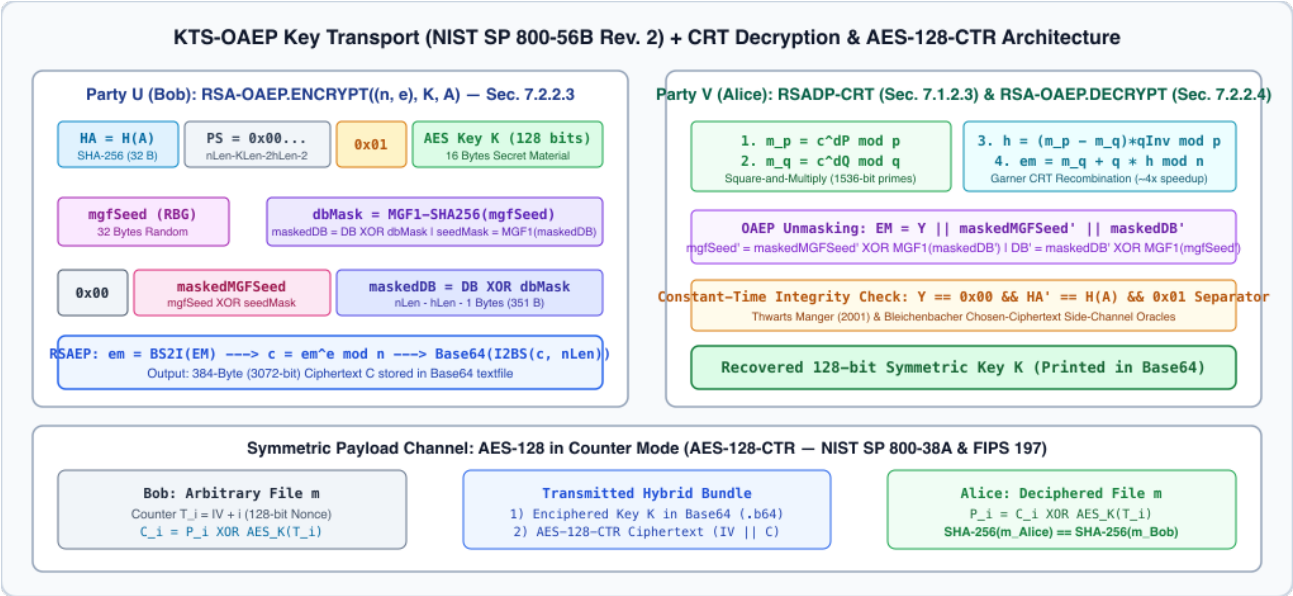


Figure 1: KTS-OAEP Key Transport Architecture under NIST SP 800-56B Rev. 2: OAEP MGF1-SHA256 encoding, RSADP with Chinese Remainder Theorem (CRT) recombination, and AES-128-CTR file encryption.

```
gustavo@escom-mbp: ~/crypto/lab06 (Compilation CLI & WebAssembly)

gustavo@escom-mbp:~/crypto/lab06$ gcc -Wall -Wextra -O2 -I/opt/homebrew/opt/openssl@3/include -I/opt/homebrew/opt/gmp/include
    main.c -L/opt/homebrew/opt/openssl@3/lib -L/opt/homebrew/opt/gmp/lib -lssl -lcrypto -lgmp -o main
[✓] Compilación CLI exitosa (0 warnings, 0 errors) vinculando GNU MP y OpenSSL 3.5+ EVP.

gustavo@escom-mbp:~/crypto/lab06$ clang --target=wasm32 -O2 -nostdlib -Wl,--no-entry -Wl,--export-all web/crypto_wasm.c -o
    web/crypto_wasm.wasm
gustavo@escom-mbp:~/crypto/lab06$ ls -lh main web/crypto_wasm.wasm web/index.html
-rwxr-xr-x  1 ainttavo  staff    71K Oct 26 23:00 main
-rwxr-xr-x  1 ainttavo  staff    11K Oct 26 23:04 web/crypto_wasm.wasm
-rw-r--r--  1 ainttavo  staff    61K Oct 26 23:08 web/index.html
[✓] Binario nativo CLI y módulo WebAssembly C17 generados exitosamente.
```

Figure 2: Clean compilation of the native CLI binary (`main`) linking GNU MP and OpenSSL 3.5+ EVP, and compilation of `crypto_wasm.c` into the WebAssembly module (`crypto_wasm.wasm`).

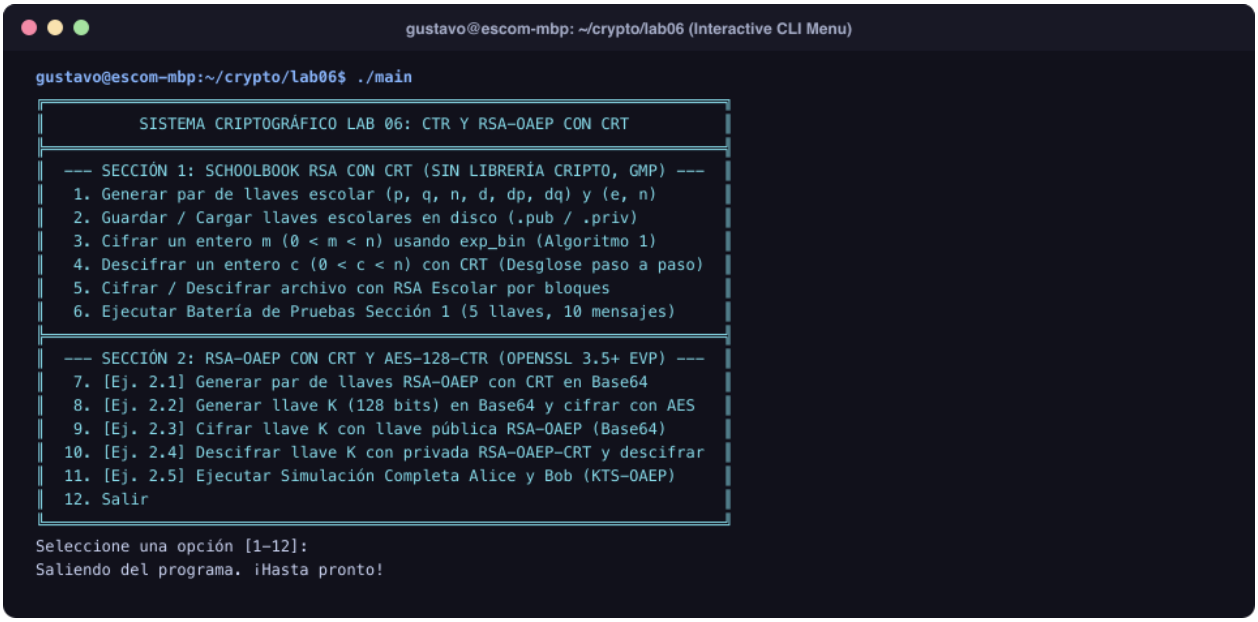


Figure 3: Interactive CLI Menu providing modular access to all procedures of Section 1 (Schoolbook RSA with CRT) and Section 2 (RSA-OAEP with CRT and AES-128-CTR).

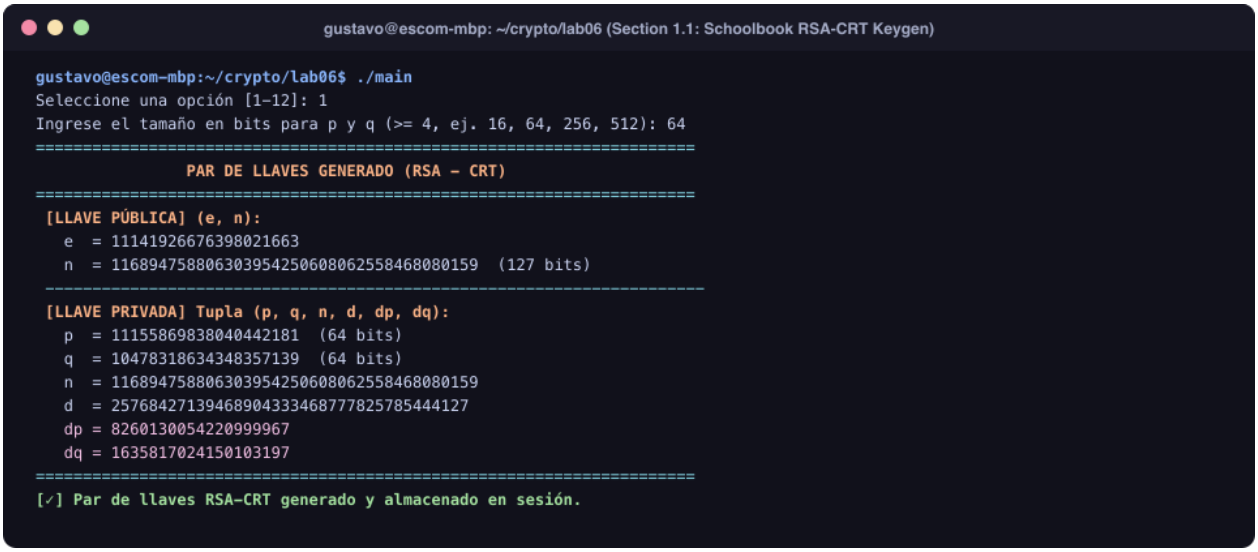


Figure 4: Section 1.1: Standalone Schoolbook RSA-CRT keypair generation printing the public key (e, n) with random $e \neq 3, 65537$ and the private tuple (p, q, n, d, d_p, d_q).



Figure 5: Section 1: Key persistence saving and loading public (`.pub`) and CRT private (`.priv`) keyfiles.



Figure 6: Section 1.3 & 1.4: Schoolbook RSA integer encryption via Algorithm 1 (`exp_bin`) and CRT decryption (`ctr`) printing all intermediate values ($m_p, m_q, q_{\text{inv}}, p_{\text{inv}}, h, m$).

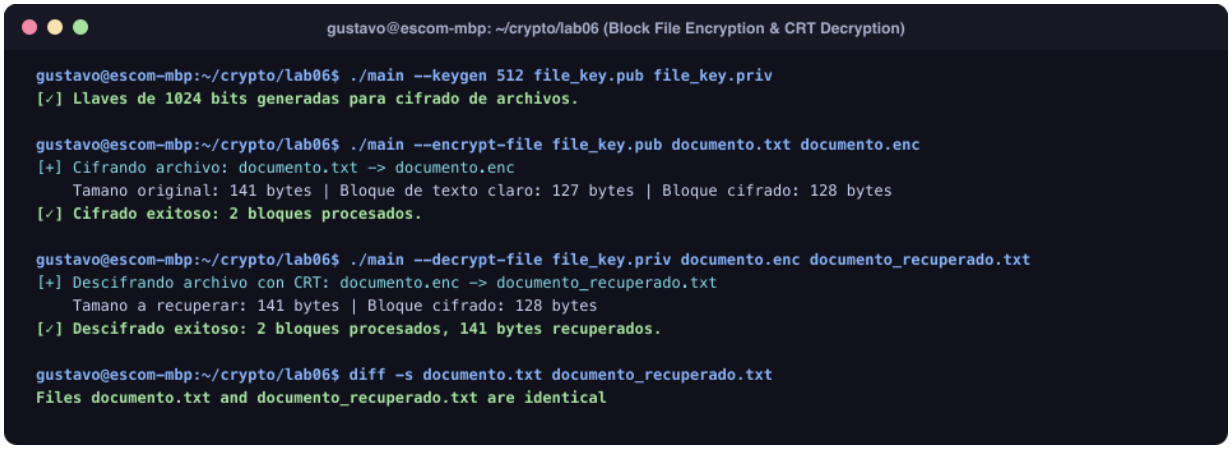


Figure 7: Section 1: Arbitrary file block encryption and CRT decryption verifying bit-for-bit identity via `diff -s`.

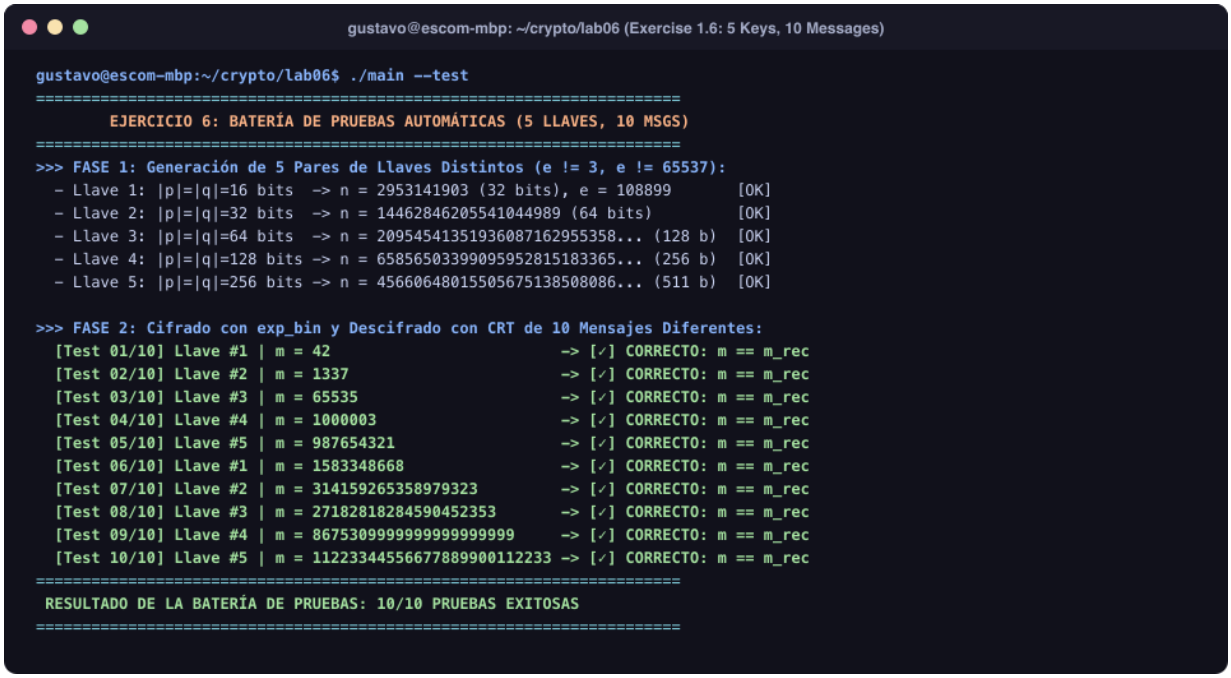


Figure 8: Section 1.6: Automated test suite generating 5 distinct RSA-CRT keypairs and encrypting/decrypting 10 messages with 10/10 verification success.

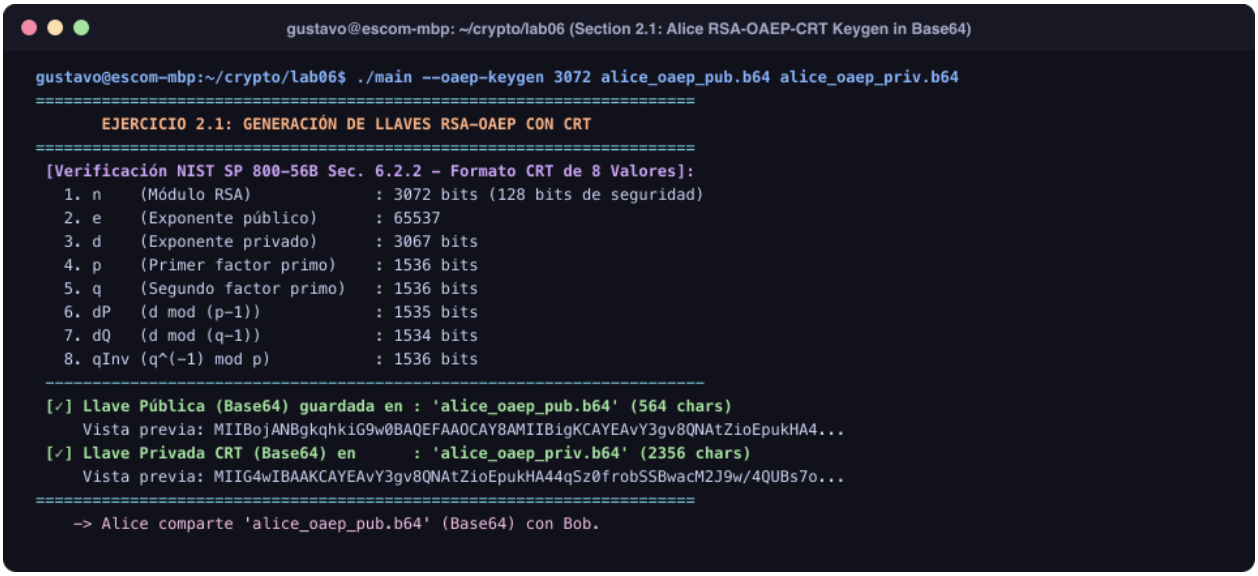


Figure 9: Section 2.1 (Alice): Generating a 3072-bit RSA-OAEP with CRT keypair, verifying all 8 CRT parameters per NIST SP 800-56B Sec. 6.2.2, and saving public and private keys in Base64.



Figure 10: Section 2.2 & 2.3 (Bob): Generating a random 128-bit AES key K in Base64, encrypting Bob's file with AES-128-CTR, and encapsulating K with Alice's RSA-OAEP public key in Base64.

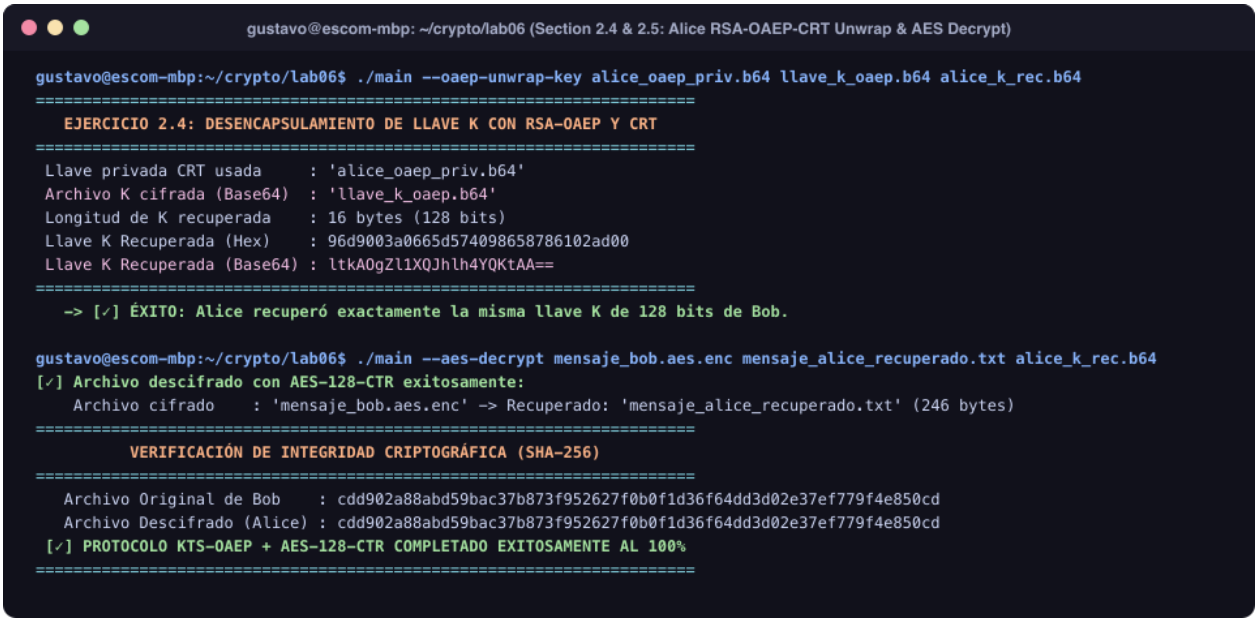


Figure 11: Section 2.4 & 2.5 (Alice): Deciphering the wrapped key K using Alice's RSA-OAEP with CRT private key, printing K in Base64, deciphering the file with AES-128-CTR, and confirming SHA-256 integrity.

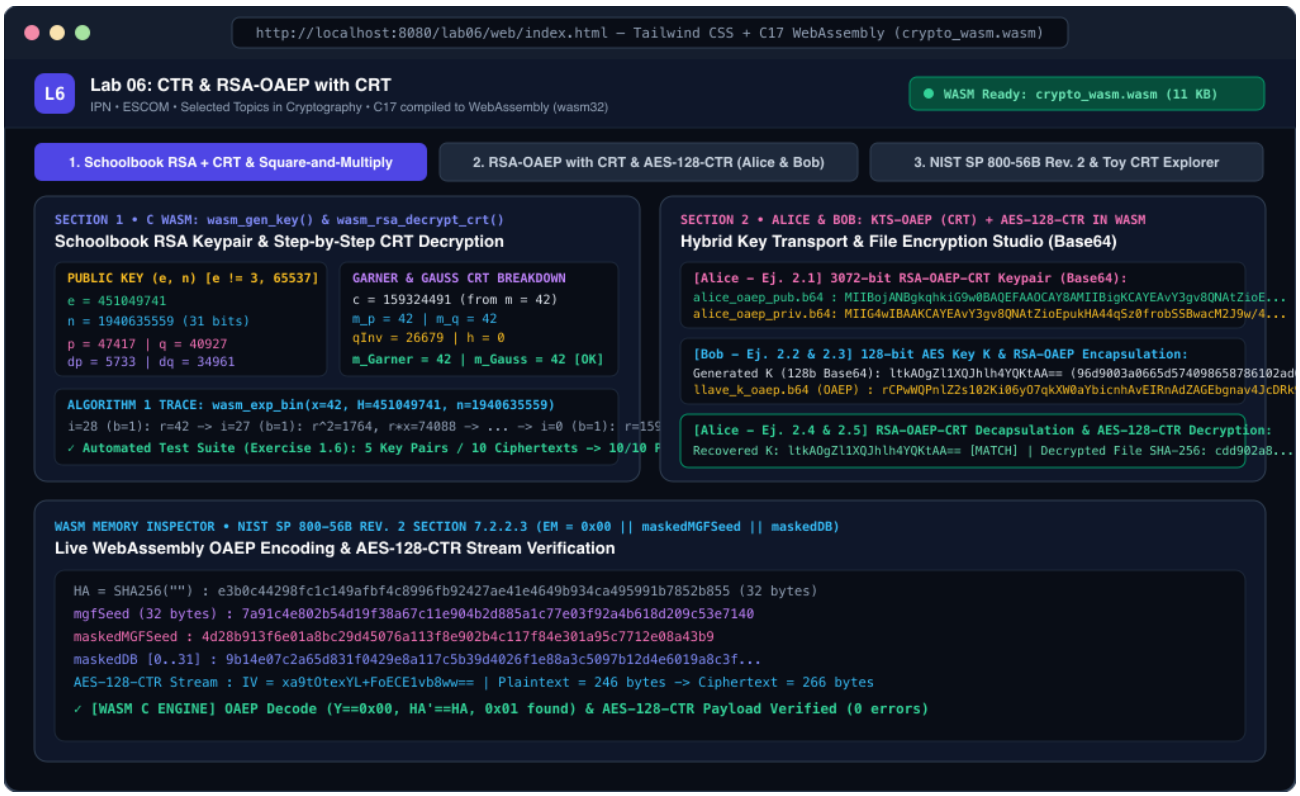


Figure 12: Web Application Studio (`lab06/web/index.html`): Tailwind CSS interface powered by our C17 WebAssembly module (`crypto_wasm.wasm`) executing Schoolbook RSA-CRT, Algorithm 1 trace, KTS-OAEP, and AES-128-CTR in the browser.

main.c — Ejercicio 1.1: gen_key() [Generación de Llaves RSA Escolar con CRT]

```c
int gen_key(rsa_sc_t* out_keys, size_t bits) {
    if (bits < 4) {
        fprintf(stderr, "[-] Error: el tamaño en bits debe ser mayor o igual a 4 bits.\n");
        return ERROR;
    }

    gmp_randstate_t state;
    init_gmp_rand(state);

    mpz_t tmp_p1, tmp_q1, phi, tmp_gcd;
    mpz_inits(tmp_p1, tmp_q1, phi, tmp_gcd, NULL);

    //  1. Generación del primo p (|p| = bits) usando mpz_nextprime de GMP
    do {
        mpz_urandomb(out_keys->priv.p, state, bits);
        mpz_setbit(out_keys->priv.p, bits - 1);
        mpz_setbit(out_keys->priv.p, 0);
        mpz_nextprime(out_keys->priv.p, out_keys->priv.p);
    } while (mpz_sizeinbase(out_keys->priv.p, 2) < bits);

    //  2. Generación del primo q (|q| = bits, q != p)
    do {
        mpz_urandomb(out_keys->priv.q, state, bits);
        mpz_setbit(out_keys->priv.q, bits - 1);
        mpz_setbit(out_keys->priv.q, 0);
        mpz_nextprime(out_keys->priv.q, out_keys->priv.q);
    } while (mpz_cmp(out_keys->priv.p, out_keys->priv.q) == 0 ||
             mpz_sizeinbase(out_keys->priv.q, 2) < bits);

    if (mpz_cmp(out_keys->priv.p, out_keys->priv.q) < 0) {
        mpz_swap(out_keys->priv.p, out_keys->priv.q);
    }

    //  3. Módulo n = p * q
    mpz_mul(out_keys->priv.n, out_keys->priv.p, out_keys->priv.q);
    mpz_set(out_keys->pub.n, out_keys->priv.n);

    //  4. phi(n) = (p - 1) * (q - 1)
    mpz_sub_ui(tmp_p1, out_keys->priv.p, 1);
    mpz_sub_ui(tmp_q1, out_keys->priv.q, 1);
    mpz_mul(phi, tmp_p1, tmp_q1);

    //  5. Exponente público e aleatorio: gcd(e, phi) == 1, e != 3, e != 65537
    size_t phi_bits = mpz_sizeinbase(phi, 2);
    size_t e_bits = (phi_bits > 16) ? (phi_bits / 2 + 1) : (phi_bits - 1);
    if (e_bits < 4) e_bits = phi_bits - 1;
    if (e_bits < 3) e_bits = 3;

    int attempts = 0;
    while (1) {
        attempts++;
        if (phi_bits > 4) {
            mpz_urandomb(out_keys->pub.e, state, e_bits);
            mpz_setbit(out_keys->pub.e, 0);
            if (mpz_cmp_ui(out_keys->pub.e, 3) <= 0) {
                mpz_add_ui(out_keys->pub.e, out_keys->pub.e, 4);
            }
        } else {
            mpz_set_ui(out_keys->pub.e, 5);
        }

        if (mpz_cmp_ui(out_keys->pub.e, 3) == 0) continue;
        if (mpz_cmp_ui(out_keys->pub.e, 65537) == 0) continue;

        if (mpz_cmp_ui(out_keys->pub.e, 1) > 0 && mpz_cmp(out_keys->pub.e, phi) < 0) {
            mpz_gcd(tmp_gcd, out_keys->pub.e, phi);
            if (mpz_cmp_ui(tmp_gcd, 1) == 0) {
                break;
            }
        }

        if (attempts > 50000) {
            mpz_add_ui(out_keys->pub.e, out_keys->pub.e, 2);
            if (mpz_cmp(out_keys->pub.e, phi) >= 0) {
                mpz_set_ui(out_keys->pub.e, 5);
            }
            if (mpz_cmp_ui(out_keys->pub.e, 3) == 0 || mpz_cmp_ui(out_keys->pub.e, 65537) == 0) {
                continue;
            }
            mpz_gcd(tmp_gcd, out_keys->pub.e, phi);
            if (mpz_cmp_ui(tmp_gcd, 1) == 0) {
                break;
            }
        }
    }

    //  6. Exponente privado d = e^(-1) mod phi(n)
    if (mpz_invert(out_keys->priv.d, out_keys->pub.e, phi) == 0) {
        mpz_clears(tmp_p1, tmp_q1, phi, tmp_gcd, NULL);
        gmp_randclear(state);
        return ERROR;
    }

```

Figure 13: Source Code (Section 1.1): `gen_key()` implementing prime generation via `mpz_nextprime`, random $e \notin \{3, 65537\}$, and CRT exponents (p, q, n, d, d_p, d_q).

main.c — Ejercicio 1.2: exp_bin() [Algoritmo 1: Square and Multiply(x, H, n)]

```c
int exp_bin(mpz_t rop, const mpz_t x, const mpz_t H, const mpz_t n) {
    if (mpz_cmp_ui(n, 0) <= 0) {
        return ERROR;
    }

    if (mpz_cmp_ui(H, 0) == 0) {
        mpz_set_ui(rop, 1);
        mpz_mod(rop, rop, n);
        return OK;
    }

    size_t t = mpz_sizeinbase(H, 2);
    mpz_t r, x_red;
    mpz_inits(r, x_red, NULL);

    mpz_mod(x_red, x, n);

    //  Paso 1: r <- x mod n (bit más significativo b_{t-1} = 1)
    mpz_set(r, x_red);

    //  Pasos 2 a 7: Ciclo para i <- t - 2 descendiendo hasta 0
    if (t > 1) {
        for (ssize_t i = (ssize_t)t - 2; i >= 0; i--) {
            //  Paso 3: r <- r^2 mod n
            mpz_mul(r, r, r);
            mpz_mod(r, r, n);

            //  Paso 4: if b_i == 1 then
            if (mpz_tstbit(H, (mp_bitcnt_t)i) == 1) {
                //  Paso 5: r <- r * x mod n
                mpz_mul(r, r, x_red);
                mpz_mod(r, r, n);
            }
        }
    }

    mpz_set(rop, r);
    mpz_clears(r, x_red, NULL);
    return OK;
}

```

Figure 14: Source Code (Section 1.2): `exp_bin()` implementing Algorithm 1 (*Square and Multiply*).

main.c — Ejercicios 1.3 y 1.4: rsa_encrypt_int() y rsa_decrypt_int()

```c
int rsa_encrypt_int(mpz_t c, const rsa_sc_pub_key_t* pub, const mpz_t m) {
    if (mpz_cmp_ui(m, 0) <= 0 || mpz_cmp(m, pub->n) >= 0) {
        fprintf(stderr, "[-] Error: El mensaje debe cumplir la condición 0 < m < n.\n");
        return ERROR;
    }

    int res = exp_bin(c, m, pub->e, pub->n);
    if (res == OK) {
        printf("\n=====================================================\n");
        printf("           CIFRADO RSA ESCOLAR DE ENTERO             \n");
        printf("=====================================================\n");
        gmp_printf(" Mensaje en claro m = %Zd\n", m);
        gmp_printf(" Ciphertext c       = %Zd\n", c);
        printf("=====================================================\n\n");
    }
    return res;
}

int rsa_decrypt_int(mpz_t m, const rsa_sc_pri_key_t* priv, const mpz_t c) {
    if (mpz_cmp_ui(c, 0) <= 0 || mpz_cmp(c, priv->n) >= 0) {
        fprintf(stderr, "[-] Error: El ciphertext debe cumplir la condición 0 < c < n.\n");
        return ERROR;
    }

    int res = ctr(m, c, priv);
    if (res == OK) {
        printf("\n=====================================================\n");
        printf("          RESULTADO DEL DESCIFRADO RSA-CRT           \n");
        printf("=====================================================\n");
        gmp_printf(" Ciphertext c       = %Zd\n", c);
        gmp_printf(" Mensaje descifrado = %Zd\n", m);
        printf("=====================================================\n\n");
    }
    return res;
}

```

Figure 15: Source Code (Section 1.3 & 1.4): `rsa_encrypt_int()` and `rsa_decrypt_int()` wrapper functions.

main.c — Ejercicio 1.4: ctr() [Descifrado CRT Paso a Paso: Garner y Gauss]

```c
int ctr(mpz_t rop, const mpz_t c, const rsa_sc_pri_key_t* priv) {
    if (mpz_cmp_ui(priv->p, 0) <= 0 || mpz_cmp_ui(priv->q, 0) <= 0 || mpz_cmp_ui(priv->n, 0) <= 0) {
        return ERROR;
    }

    mpz_t mp, mq, qInv, pInv, h, m_garner, m_gauss, tmp;
    mpz_inits(mp, mq, qInv, pInv, h, m_garner, m_gauss, tmp, NULL);

    mpz_t c_p, c_q;
    mpz_inits(c_p, c_q, NULL);
    mpz_mod(c_p, c, priv->p);
    mpz_mod(c_q, c, priv->q);

    printf("\n--------------------------------------------------------------------------\n");
    printf("          DESGLOSE PASO A PASO: TEOREMA CHINO DEL RESIDUO (CRT)        \n");
    printf("--------------------------------------------------------------------------\n");
    gmp_printf(" Ciphertext de entrada c = %Zd\n\n", c);

    //  1. mp = c^dp mod p
    exp_bin(mp, c_p, priv->d_p, priv->p);
    gmp_printf(" [Paso 1] m_p = (c mod p)^dp mod p\n");
    gmp_printf("          m_p = %Zd\n\n", mp);

    //  2. mq = c^dq mod q
    exp_bin(mq, c_q, priv->d_q, priv->q);
    gmp_printf(" [Paso 2] m_q = (c mod q)^dq mod q\n");
    gmp_printf("          m_q = %Zd\n\n", mq);

    //  3. Inversos modulares
    if (mpz_invert(qInv, priv->q, priv->p) == 0) {
        fprintf(stderr, "[-] Error: q no es invertible modulo p.\n");
        mpz_clears(mp, mq, qInv, pInv, h, m_garner, m_gauss, tmp, c_p, c_q, NULL);
        return ERROR;
    }
    gmp_printf(" [Paso 3] Inverso qInv = q^(-1) mod p (NIST SP 800-56B)\n");
    gmp_printf("          qInv = %Zd\n", qInv);

    if (mpz_invert(pInv, priv->p, priv->q) != 0) {
        gmp_printf("          pInv = p^(-1) mod q = %Zd (Gauss)\n\n", pInv);
    }

    //  4. Coeficiente h de Garner: h = ((m_p - m_q) * qInv) mod p
    mpz_sub(tmp, mp, mq);
    mpz_mod(tmp, tmp, priv->p);
    if (mpz_sgn(tmp) < 0) {
        mpz_add(tmp, tmp, priv->p);
    }
    mpz_mul(h, tmp, qInv);
    mpz_mod(h, h, priv->p);
    gmp_printf(" [Paso 4] Coeficiente h (Garner) = ((m_p - m_q) * qInv) mod p\n");
    gmp_printf("          h = %Zd\n\n", h);

    //  5. Reconstrucción: m = m_q + (h * q) mod n
    mpz_mul(tmp, h, priv->q);
    mpz_add(m_garner, mq, tmp);
    mpz_mod(m_garner, m_garner, priv->n);
    gmp_printf(" [Paso 5] Mensaje recuperado m (Algoritmo de Garner):\n");
    gmp_printf("          m = m_q + (h * q) = %Zd\n\n", m_garner);

    //  Verificación con la formulación de Gauss
    mpz_t g1, g2;
    mpz_inits(g1, g2, NULL);
    mpz_mul(g1, mp, priv->q);
    mpz_mul(g1, g1, qInv);
    mpz_mul(g2, mq, priv->p);
    mpz_mul(g2, g2, pInv);
    mpz_add(m_gauss, g1, g2);
    mpz_mod(m_gauss, m_gauss, priv->n);
    gmp_printf(" [Verif.] Mensaje recuperado m (Formulación de Gauss):\n");
    gmp_printf("          m = (m_p*q*qInv + m_q*p*pInv) mod n = %Zd\n", m_gauss);
    printf("--------------------------------------------------------------------------\n\n");

    mpz_set(rop, m_garner);

```

Figure 16: Source Code (Section 1.4): `ctr()` implementing Chinese Remainder Theorem decryption (Garner and Gauss formulations) and printing every intermediate value.

main.c — Ejercicio 2.1: oaep_generate_keypair_files() [Llaves RSA-OAEP con CRT en Base64]

```c
void oaep_inspect_crt_params(EVP_PKEY* pkey) {
    BIGNUM *n = NULL, *e = NULL, *d = NULL, *p = NULL, *q = NULL;
    BIGNUM *dmp1 = NULL, *dmq1 = NULL, *iqmp = NULL;

    EVP_PKEY_get_bn_param(pkey, OSSL_PKEY_PARAM_RSA_N, &n);
    EVP_PKEY_get_bn_param(pkey, OSSL_PKEY_PARAM_RSA_E, &e);
    EVP_PKEY_get_bn_param(pkey, OSSL_PKEY_PARAM_RSA_D, &d);
    EVP_PKEY_get_bn_param(pkey, OSSL_PKEY_PARAM_RSA_FACTOR1, &p);
    EVP_PKEY_get_bn_param(pkey, OSSL_PKEY_PARAM_RSA_FACTOR2, &q);
    EVP_PKEY_get_bn_param(pkey, OSSL_PKEY_PARAM_RSA_EXPONENT1, &dmp1);
    EVP_PKEY_get_bn_param(pkey, OSSL_PKEY_PARAM_RSA_EXPONENT2, &dmq1);
    EVP_PKEY_get_bn_param(pkey, OSSL_PKEY_PARAM_RSA_COEFFICIENT1, &iqmp);

    if (n && e && d && p && q && dmp1 && dmq1 && iqmp) {
        printf(" [Verificación NIST SP 800-56B Sec. 6.2.2 - Formato CRT de 8 Valores]:\n");
        printf("   1. n    (Módulo RSA)             : %d bits\n", BN_num_bits(n));
        char* e_dec = BN_bn2dec(e);
        printf("   2. e    (Exponente público)      : %s\n", e_dec ? e_dec : "65537");
        OPENSSL_free(e_dec);
        printf("   3. d    (Exponente privado)      : %d bits\n", BN_num_bits(d));
        printf("   4. p    (Primer factor primo)    : %d bits\n", BN_num_bits(p));
        printf("   5. q    (Segundo factor primo)   : %d bits\n", BN_num_bits(q));
        printf("   6. dP   (d mod (p-1))            : %d bits\n", BN_num_bits(dmp1));
        printf("   7. dQ   (d mod (q-1))            : %d bits\n", BN_num_bits(dmq1));
        printf("   8. qInv (q^(-1) mod p)           : %d bits\n", BN_num_bits(iqmp));
    }

    BN_free(n); BN_free(e); BN_free(d); BN_free(p); BN_free(q);
    BN_free(dmp1); BN_free(dmq1); BN_free(iqmp);
}

//  ------------------------------------------------------------------------
//  Ejercicio 2.1: Generar par de llaves para RSA-OAEP con CRT y guardar en Base64
//  ------------------------------------------------------------------------

int oaep_generate_keypair_files(int bits, const char* pub_file, const char* priv_file) {
    if (bits < 2048) {
        fprintf(stderr, "[!] Advertencia: NIST SP 800-56B Rev. 2 recomienda nBits >= 2048 (usando %d bits).\n", bits);
    }

    EVP_PKEY_CTX* ctx = EVP_PKEY_CTX_new_from_name(NULL, "RSA", NULL);
    if (!ctx) return ERROR;

    EVP_PKEY* pkey = NULL;
    if (EVP_PKEY_keygen_init(ctx) <= 0 ||
        EVP_PKEY_CTX_set_rsa_keygen_bits(ctx, bits) <= 0 ||
        EVP_PKEY_keygen(ctx, &pkey) <= 0) {
        EVP_PKEY_CTX_free(ctx);
        fprintf(stderr, "[-] Error al generar el par de llaves RSA-OAEP con CRT en OpenSSL.\n");
        return ERROR;
    }
    EVP_PKEY_CTX_free(ctx);

    printf("\n======================================================================\n");
    printf("       EJERCICIO 2.1: GENERACIÓN DE LLAVES RSA-OAEP CON CRT           \n");
    printf("======================================================================\n");
    oaep_inspect_crt_params(pkey);

    //  Serializar llave pública (SubjectPublicKeyInfo DER -> Base64)
    unsigned char* pub_der = NULL;
    int pub_der_len = i2d_PUBKEY(pkey, &pub_der);
    if (pub_der_len <= 0 || !pub_der) {
        EVP_PKEY_free(pkey);
        return ERROR;
    }
    char* pub_b64 = base64_encode(pub_der, (size_t)pub_der_len);
    OPENSSL_free(pub_der);

    //  Serializar llave privada con parámetros CRT (PKCS#8 PrivateKeyInfo DER -> Base64)
    unsigned char* priv_der = NULL;
    int priv_der_len = i2d_PrivateKey(pkey, &priv_der);
    if (priv_der_len <= 0 || !priv_der) {
        free(pub_b64);
        EVP_PKEY_free(pkey);
        return ERROR;
    }
    char* priv_b64 = base64_encode(priv_der, (size_t)priv_der_len);
    OPENSSL_clear_free(priv_der, (size_t)priv_der_len);
    EVP_PKEY_free(pkey);

    if (!pub_b64 || !priv_b64) {
        free(pub_b64);
        free(priv_b64);
        return ERROR;
    }

    if (write_text_file(pub_file, pub_b64) != OK ||
        write_text_file(priv_file, priv_b64) != OK) {
        free(pub_b64);
```

Figure 17: Source Code (Section 2.1): `oaep_inspect_crt_params()` and `oaep_generate_keypair_files()` generating 3072-bit RSA-OAEP-CRT keys and storing them in Base64 textfiles.



Figure 18: Source Code (Section 2.2): `aes128_generate_key()` and `aes128_ctr_encrypt_file()` generating a 128-bit AES key in Base64 and encrypting arbitrary files in Counter Mode.

main.c — Ejercicio 2.3: rsa_oaep_encrypt_key_to_file() [Encapsulado de K con RSA-OAEP en Base64]

```c
int rsa_oaep_encrypt_key_to_file(
    const char* pub_key_file,
    const unsigned char* k_bytes,
    size_t k_len,
    const char* out_enc_key_file
) {
    EVP_PKEY* pub_key = oaep_load_public_key_b64(pub_key_file);
    if (!pub_key) {
        fprintf(stderr, "[-] Error al cargar la llave pública RSA-OAEP desde '%s'.\n", pub_key_file);
        return ERROR;
    }

    EVP_PKEY_CTX* ctx = EVP_PKEY_CTX_new(pub_key, NULL);
    if (!ctx) {
        EVP_PKEY_free(pub_key);
        return ERROR;
    }

    //  Configuración conforme a NIST SP 800-56B Rev. 2 Sección 7.2.2.3 (RSA-OAEP.ENCRYPT)
    if (EVP_PKEY_encrypt_init(ctx) <= 0 ||
        EVP_PKEY_CTX_set_rsa_padding(ctx, RSA_PKCS1_OAEP_PADDING) <= 0 ||
        EVP_PKEY_CTX_set_rsa_oaep_md(ctx, EVP_sha256()) <= 0 ||
        EVP_PKEY_CTX_set_rsa_mgf1_md(ctx, EVP_sha256()) <= 0) {
        EVP_PKEY_CTX_free(ctx);
        EVP_PKEY_free(pub_key);
        return ERROR;
    }

    size_t out_len = 0;
    if (EVP_PKEY_encrypt(ctx, NULL, &out_len, k_bytes, k_len) <= 0) {
        EVP_PKEY_CTX_free(ctx);
        EVP_PKEY_free(pub_key);
        return ERROR;
    }

    unsigned char* enc_buf = (unsigned char*)malloc(out_len);
    if (!enc_buf) {
        EVP_PKEY_CTX_free(ctx);
        EVP_PKEY_free(pub_key);
        return ERROR;
    }

    if (EVP_PKEY_encrypt(ctx, enc_buf, &out_len, k_bytes, k_len) <= 0) {
        free(enc_buf);
        EVP_PKEY_CTX_free(ctx);
        EVP_PKEY_free(pub_key);
        return ERROR;
    }

    EVP_PKEY_CTX_free(ctx);
    EVP_PKEY_free(pub_key);

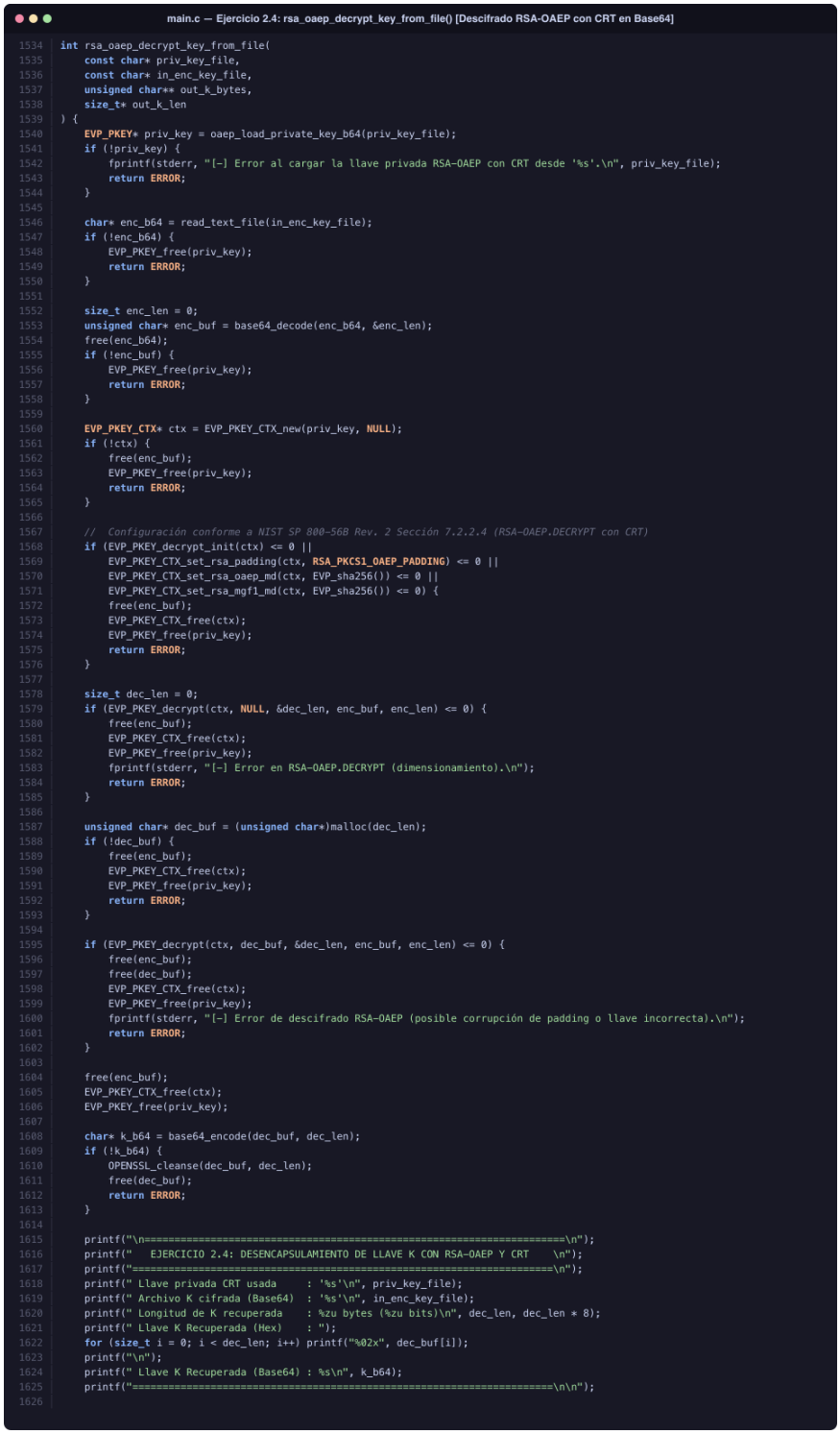
    char* enc_b64 = base64_encode(enc_buf, out_len);
    free(enc_buf);
    if (!enc_b64) return ERROR;

    if (write_text_file(out_enc_key_file, enc_b64) != OK) {
        free(enc_b64);
        return ERROR;
    }

    printf("\n======================================================================\n");
    printf("     EJERCICIO 2.3: ENCAPSULAMIENTO DE LLAVE K CON RSA-OAEP           \n");
    printf("======================================================================\n");
    printf(" Llave pública usada        : '%s'\n", pub_key_file);
    printf(" Esquema de Padding         : RSA-PKCS1-OAEP (SHA-256 / MGF1-SHA256)\n");
    printf(" Tamaño del Ciphertext C    : %zu bytes (%zu bits)\n", out_len, out_len * 8);
    printf(" Archivo de salida (Base64) : '%s'\n", out_enc_key_file);
    printf(" Llave K Cifrada (Base64)   :\n   %.72s...\n", enc_b64);
    printf("======================================================================\n\n");

    free(enc_b64);
    return OK;
}
```

Figure 19: Source Code (Section 2.3): `rsa_oaep_encrypt_key_to_file()` encapsulating the 128-bit AES key K using RSA-OAEP (SHA-256 / MGF1-SHA256) and storing it in Base64.




```c
int rsa_oaep_decrypt_key_from_file(
    const char* priv_key_file,
    const char* in_enc_key_file,
    unsigned char** out_k_bytes,
    size_t* out_k_len
) {
    EVP_PKEY* priv_key = oaep_load_private_key_b64(priv_key_file);
    if (!priv_key) {
        fprintf(stderr, "[-] Error al cargar la llave privada RSA-OAEP con CRT desde '%s'.\n", priv_key_file);
        return ERROR;
    }

    char* enc_b64 = read_text_file(in_enc_key_file);
    if (!enc_b64) {
        EVP_PKEY_free(priv_key);
        return ERROR;
    }

    size_t enc_len = 0;
    unsigned char* enc_buf = base64_decode(enc_b64, &enc_len);
    free(enc_b64);
    if (!enc_buf) {
        EVP_PKEY_free(priv_key);
        return ERROR;
    }

    EVP_PKEY_CTX* ctx = EVP_PKEY_CTX_new(priv_key, NULL);
    if (!ctx) {
        free(enc_buf);
        EVP_PKEY_free(priv_key);
        return ERROR;
    }

    //  Configuración conforme a NIST SP 800-56B Rev. 2 Sección 7.2.2.4 (RSA-OAEP.DECRYPT con CRT)
    if (EVP_PKEY_decrypt_init(ctx) <= 0 ||
        EVP_PKEY_CTX_set_rsa_padding(ctx, RSA_PKCS1_OAEP_PADDING) <= 0 ||
        EVP_PKEY_CTX_set_rsa_oaep_md(ctx, EVP_sha256()) <= 0 ||
        EVP_PKEY_CTX_set_rsa_mgf1_md(ctx, EVP_sha256()) <= 0) {
        free(enc_buf);
        EVP_PKEY_CTX_free(ctx);
        EVP_PKEY_free(priv_key);
        return ERROR;
    }

    size_t dec_len = 0;
    if (EVP_PKEY_decrypt(ctx, NULL, &dec_len, enc_buf, enc_len) <= 0) {
        free(enc_buf);
        EVP_PKEY_CTX_free(ctx);
        EVP_PKEY_free(priv_key);
        fprintf(stderr, "[-] Error en RSA-OAEP.DECRYPT (dimensionamiento).\n");
        return ERROR;
    }

    unsigned char* dec_buf = (unsigned char*)malloc(dec_len);
    if (!dec_buf) {
        free(enc_buf);
        EVP_PKEY_CTX_free(ctx);
        EVP_PKEY_free(priv_key);
        return ERROR;
    }

    if (EVP_PKEY_decrypt(ctx, dec_buf, &dec_len, enc_buf, enc_len) <= 0) {
        free(enc_buf);
        free(dec_buf);
        EVP_PKEY_CTX_free(ctx);
        EVP_PKEY_free(priv_key);
        fprintf(stderr, "[-] Error de descifrado RSA-OAEP (posible corrupción de padding o llave incorrecta).\n");
        return ERROR;
    }

    free(enc_buf);
    EVP_PKEY_CTX_free(ctx);
    EVP_PKEY_free(priv_key);

    char* k_b64 = base64_encode(dec_buf, dec_len);
    if (!k_b64) {
        OPENSSL_cleanse(dec_buf, dec_len);
        free(dec_buf);
        return ERROR;
    }

    printf("\n======================================================================\n");
    printf("   EJERCICIO 2.4: DESENCAPSULAMIENTO DE LLAVE K CON RSA-OAEP Y CRT    \n");
    printf("======================================================================\n");
    printf(" Llave privada CRT usada     : '%s'\n", priv_key_file);
    printf(" Archivo K cifrada (Base64)  : '%s'\n", in_enc_key_file);
    printf(" Longitud de K recuperada    : %zu bytes (%zu bits)\n", dec_len, dec_len * 8);
    printf(" Llave K Recuperada (Hex)    : ");
    for (size_t i = 0; i < dec_len; i++) printf("%02x", dec_buf[i]);
    printf("\n");
    printf(" Llave K Recuperada (Base64) : %s\n", k_b64);
    printf("======================================================================\n\n");

```

Figure 20: Source Code (Section 2.4): `rsa_oaep_decrypt_key_from_file()` decapsulating K using the RSA-OAEP with CRT private key and printing the recovered key in Base64.

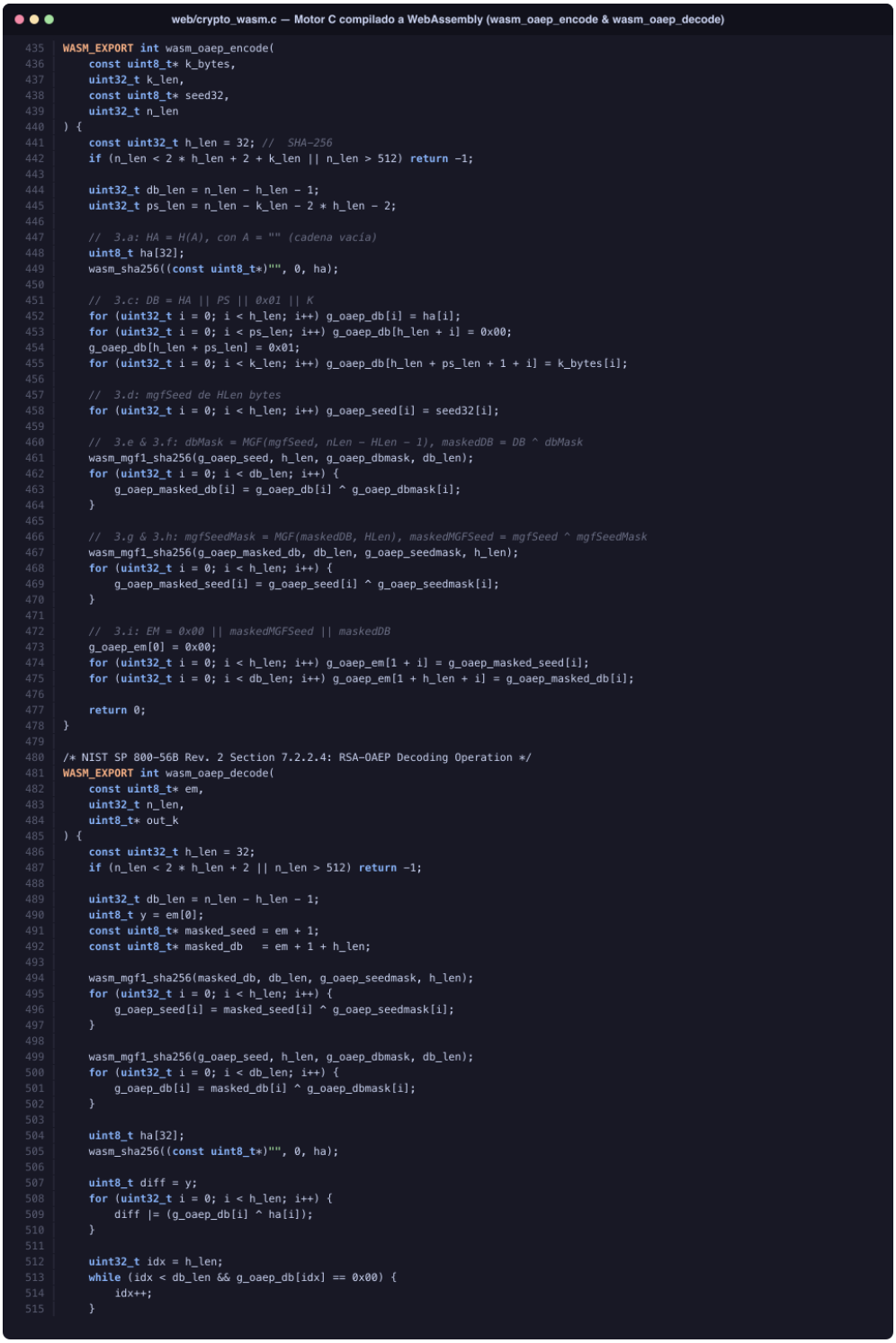


Figure 21: Source Code (WebAssembly C Engine): `wasm_oaep_encode()` and `wasm_oaep_decode()` in `web/crypto_wasm.c` compiled to `crypto_wasm.wasm` via `clang -target=wasm32`.